Measuring and Improving Multi-Target Binary Selection in LLVM's GPU Offload Stack

S. Akash *IIT Patna · CERN GSoC alumnus · vLLM contributor*

Thesis

MLIR can compile a single `gpu.module` to N GPU targets (NVIDIA, AMD, Intel), and `OffloadBinary` can carry all N images in one fat binary. But at runtime, LLVM's offload stack picks the *first compatible image* and stops—there is no standard metadata for "best-compatible" selection, and no published measurement of what this costs.

Contributions

1. **Standard metadata vocabulary** for `OffloadBinary`: 5 new keys enabling runtime selection beyond triple/arch matching (additive, backward-compatible, ∼30 LOC header patch).
2. **First published flame graph** of the LLVM GPU dispatch stack: per-layer latency from `OffloadBinary` parse through `cuLaunchKernel` on a GTX 1650 (10,000 iterations, 100 cold trials).
3. **Design sketch** for `#gpu.runtime_select`: an MLIR attribute deferring binary selection from compile time to runtime.

Dispatch Latency Breakdown

Layer	Median	
`cuModuleLoadData` (cold)	42.7	μs
`cuModuleLoadData` (warm)	10.0	μs
`cuModuleGetFunction`	60	ns
`cuLaunchKernel` (submit)	1.6	μs
Hot-path (launch + sync)	4.0	μs
Binary selection logic	6	ns

GTX 1650 (sm_75), null kernel, CUBIN pre-compiled. Cold = `execve`-isolated trials. Selection = variant table scan.

MLIR Syntax: `#gpu.runtime_select`

```
gpu.binary @kernels [
  #gpu.object<#nvptx, "sm_75.cubin">,
  #gpu.object<#amdgcn, "gfx90a.hsaco">
] { select = #gpu.runtime_select }
```

OffloadBinary Metadata (5 New Keys)

Key	Tier	Semantics
`min_sm`	MUST	Min CUDA compute cap
`min_gfx`	MUST	Min AMD GFX + family
`requires_features`	MUST	Capability tokens
`variant_priority`	MAY	Ranking (higher = preferred)
`variant_tag`	MAY	Label: `optimized`, `fallback`

MUST = runtime rejects if violated. MAY = ranking only. Missing keys = no constraint (backward-compatible).

Links

[QR: GitHub]	Source, benchmarks, patch
[QR: Discourse RFC]	RFC discussion thread